MSML 605

Concurrency

Thanks to Manoj Franklin for the slides' content

Code examples modified by Mohammad Nayeem Teli

Outline

- *Introduction*
- *os.fork Function*
- *os.system Function and os.exec Family of Functions*
- *Controlling Process Input and Output*
- *Interprocess Communication*
- *Signal Handling*
- *Sending Signals*

Introduction

- Implement concurrency by having each task (or process) operate in a separate memory space
- Time slicing on single-processor systems divides processor time among many processes
- OS allocates a short execution time called a quantum to a process
- OS performs context switching to move processes and their dependent data in and out of memory

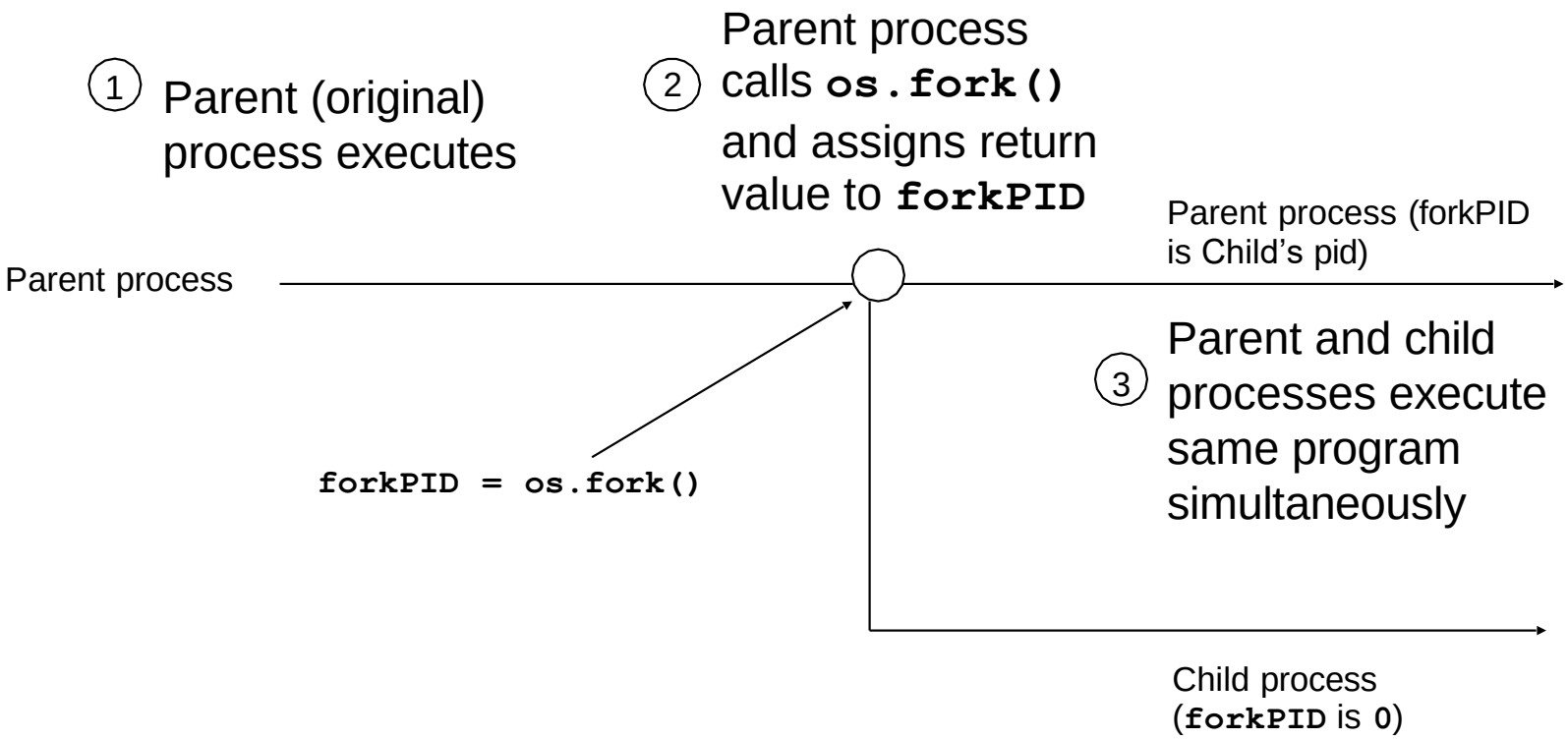
Introduction

- OS offers shells – which are software programs that carry out system commands for users.
- Certain operating systems come equipped with system commands that allow programmers to develop and control processes.
  - For instance, the POSIX (Portable Operating System Interface for UNIX) standard outlines these system commands specifically for UNIX-based operating systems.

os Module

- `os` module provides functions for interacting with the operating system (OS)
- Provides a portable way of using OS dependent functionality
- Every function within the **os.fork** module will raise an **OSError** if provided with invalid or inaccessible file names, paths, or other correct but unacceptable arguments.

`os.fork()` Function



`os.fork()` creates a new process

```
1    # pid.py
2    # Using fork to create child processes.
3
4    import os
5    import sys
6    import time
7
8    processName = "parent"  # only the parent is running now
9
10   print ("Program executing\n\tpid: %d, processName: %s" \
11       % ( os.getpid(), processName ))
12
13   # attempt to fork child process
14   try:
15       forkPID = os.fork()  # create child process
16   except OSError:
17       sys.exit( "Unable to create new process." )
18
19   if forkPID != 0:  # am I parent process?
20       print ("Parent executing\n" + \
21           "\tpid: %d, forkPID: %d, processName: %s" \
22           % ( os.getpid(), forkPID, processName ))
23
24   elif forkPID == 0:  # am I child process?
25       processName = "child"
26       print ("Child executing\n" + \
27           "\tpid: %d, forkPID: %d, processName: %s" \
28           % ( os.getpid(), forkPID, processName ))
29
30   print ("Process finishing\n\tpid: %d, processName: %s" \
31       % ( os.getpid(), processName ))
```

Creates duplicate of current process

`os.fork` raises **`OSError`** exception if fork failed

Parent process

Child process has *pid* of 0

```
Program executing
    pid: 6537, processName: parent
Parent executing
    pid: 6537, forkPID: 6538, processName: parent
Process finishing
    pid: 6537, processName: parent
Child executing
    pid: 6538, forkPID: 0, processName: child
Process finishing
    pid: 6538, processName: child
```

```
Program executing
    pid: 6664, processName: parent
Parent executing
    pid: 6664, forkPID: 6665, processName: parent
Process finishing
    pid: 6664, processName: parent
Child executing
    pid: 6665, forkPID: 0, processName: child
Process finishing
    pid: 6665, processName: child
```

```
# wait.py
# Demonstrates the wait function.

import os
import sys
import time
import random

# generate random sleep times for child processes
sleepTime1 = random.randrange( 1, 6 )
sleepTime2 = random.randrange( 1, 6 )

# parent ready to fork first child process
try:
   forkPID1 = os.fork()  # create first child process
except OSError:
   sys.exit( "Unable to create first child. " )

if forkPID1 != 0:  # am I parent process?

   # parent ready to fork second child process
   try:
      forkPID2 = os.fork()  # create second child process
   except OSError:
      sys.exit( "Unable to create second child." )

   if forkPID2 != 0:  # am I parent process?
      print ("Parent waiting for child processes...\n" + \
             "\tpid: %d, forkPID1: %d, forkPID2: %d" \
             % ( os.getpid(), forkPID1, forkPID2 ))
```

Generate random sleep times for child processes

Create first child process

Create second child process

Function **`os.wait`** causes parent to wait for child process to complete execution

```
      # wait for any child process
      try:
         child1 = os.wait()[ 0 ]   # wait returns one child's pid
      except OSError:
         sys.exit( "No more child processes." )

      print ("Parent: Child %d finished first, one child left." % child1)

      # wait for another child process
      try:
         child2 = os.wait()[ 0 ]   # wait returns other child's pid
      except OSError:
         sys.exit( "No more child processes." )

      print ("Parent: Child %d finished second, no children left." % child2)

   elif forkPID2 == 0:   # am I second child process?
      print ("""Child2 sleeping for %d seconds...
             \tpid: %d, forkPID1: %d, forkPID2: %d""" \
             % ( sleepTime2, os.getpid(), forkPID1, forkPID2 ))
      time.sleep( sleepTime2 )   # sleep to simulate some work

elif forkPID1 == 0:   # am I first child process?
   print ("""Child1 sleeping for %d seconds...
          \tpid: %d, forkPID1: %d""" \
          % ( sleepTime1, os.getpid(), forkPID1 ))
   time.sleep( sleepTime1 )   # sleep to simulate some work
```

Function **`os.wait`** returns finished child's *pid*

```
Parent waiting for child processes...
    pid: 6887, forkPID1: 6888, forkPID2: 6889
Child1 sleeping for 3 seconds...
            pid: 6888, forkPID1: 0
Child2 sleeping for 3 seconds...
                pid: 6889, forkPID1: 6888, forkPID2: 0
Parent: Child 6888 finished first, one child left.
Parent: Child 6889 finished second, no children left.
```

```
Parent waiting for child processes...
    pid: 6898, forkPID1: 6899, forkPID2: 6900
Child1 sleeping for 1 seconds...
            pid: 6899, forkPID1: 0
Child2 sleeping for 3 seconds...
                pid: 6900, forkPID1: 6899, forkPID2: 0
Parent: Child 6899 finished first, one child left.
Parent: Child 6900 finished second, no children left.
```

```
Parent waiting for child processes...
    pid: 6901, forkPID1: 6902, forkPID2: 6903
Child1 sleeping for 3 seconds...
            pid: 6902, forkPID1: 0
Child2 sleeping for 3 seconds...
                pid: 6903, forkPID1: 6902, forkPID2: 0
Parent: Child 6903 finished first, one child left.
Parent: Child 6902 finished second, no children left.
```

```
# waitpid.py
# Demonstrates the waitpid function.

import os
import sys
import time

# parent about to fork first child process
try:
   forkPID1 = os.fork()   # create first child process
except OSError:
   sys.exit( "Unable to create first child. " )

if forkPID1 != 0:   # am I parent process?

   # parent about to fork second child process
   try:
      forkPID2 = os.fork()   # create second child process
   except OSError:
      sys.exit( "Unable to create second child." )
```

Create first child process

Create second child process

```
    if forkPID2 > 0:   # am I parent process?
       print "Parent waiting for child processes...\n" + \
          "\tpid: %d, forkPID1: %d, forkPID2: %d" \
            % ( os.getpid(), forkPID1, forkPID2 )

       # wait for second child process explicitly
       try:
          child2 = os.waitpid(forkPID2, 0 )[ 0 ]  # child's pid
       except OSError:
          sys.exit("No child process with pid %d." % (forkPID2 ))

       print "Parent: Child %d finished." % child2

    elif forkPID2 == 0:   # am I second child process?
       print "Child2 sleeping for 4 seconds...\n" + \
            "\tpid: %d, forkPID1: %d, forkPID2: %d" \
           % ( os.getpid(), forkPID1, forkPID2 )
       time.sleep( 4 )

elif forkPID1 == 0:   # am I first child process?
   print "Child1 sleeping for 2 seconds...\n" + \
        "\tpid: %d, forkPID1: %d" % ( os.getpid(), forkPID1 )
   time.sleep( 2 )
```

```
Parent waiting for child processes...
        pid: 7084, forkPID1: 7089, forkPID2: 7090
Child1 sleeping for 2 seconds...
        pid: 7089, forkPID1: 0
Child2 sleeping for 4 seconds...
        pid: 7090, forkPID1: 7089, forkPID2: 0
Parent: Child 7090 finished.
```

```
Parent waiting for child processes...
        pid: 7094, forkPID1: 7095, forkPID2: 7096
Child1 sleeping for 2 seconds...
        pid: 7095, forkPID1: 0
Child2 sleeping for 4 seconds...
        pid: 7096, forkPID1: 7095, forkPID2: 0
Parent: Child 7096 finished.
```

`os.system()` Function

- Function **`os.system()`** executes a system command using a shell and then returns control to the original process (after the command completes)
- This method is implemented by calling the Standard C function **`system()`**
- If command generates any output, it is sent to the interpreter standard output stream

os.system() Function

```
# systemfunc.py
# Uses the system function to clear the screen.

import os
import sys

def printInstructions( clearCommand ):
    os.system( clearCommand )   # clear display

    print """Type the text that you wish to save in this file.
    Type clear on a blank line to delete the contents of the file.
    Type quit on a blank line when you are fin

# determine operating system
if os.name == "nt" or os.name == "dos":  # Windows system
    clearCommand = "cls"
    print "You are using a Windows system."
elif os.name == "posix":   # UNIX-compatible system
    clearCommand = "clear"
    print "You are using a UNIX-compatible system."
else:
    sys.exit( "Unsupported OS" )

filename = raw_input( "What file would you like to create? " )
```

Function **`os.system`** executes system-specific clear command

Clear command depends on OS

os.system() Function

```
# open file
try:
   file = open( filename, "w+" )
except IOError, message:
   sys.exit( "Error creating file: %s" % message )

printInstructions( clearCommand )
currentLine = ""

# write input to file
while currentLine != "quit\n":
   file.write( currentLine )
   currentLine = sys.stdin.readline()

   if currentLine == "clear\n":

      # seek to beginning and truncate file
      file.seek( 0, 0 )
      file.truncate()

      currentLine = ""
      printInstructions( clearCommand )

file.close()
```

os.system() Function

```
You are using a UNIX-compatible system.
What file would you like to create?
```

```
Type the text that you wish to save in this file.
   Type clear on a blank line to delete the contents of the file.
   Type quit on a blank line when you are finished.
```

`os.exec` Family of Functions

- **`os.exec`** family of functions do not return control to calling process after executing the specified command

os.exec Family of Functions

```
python webpage.py http://www.cs.umd.edu cs.txt
```

```
<!DOCTYPE html>
<html lang="en" dir="ltr" prefix="content: http://purl.org/rss/1.0/modules/content/ dc: http://purl.org/dc/terms/ foaf:
http://xmlns.com/foaf/0.1/ rdfs: http://www.w3.org/2000/01/rdf-schema# sioc: http://rdfs.org/sioc/ns# sioct: http://
rdfs.org/sioc/types# skos: http://www.w3.org/2004/02/skos/core# xsd: http://www.w3.org/2001/XMLSchema#">
<head>
  <link rel="profile" href="http://www.w3.org/1999/xhtml/vocab" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<link rel="shortcut icon" href="https://www.cs.umd.edu/sites/all/themes/cs_gogo/favicon.ico" type="image/vnd.microsoft.icon"
/>
<meta name="description" content="Homepage of the University of Maryland&#039;s Department of Computer Science" />
<link rel="image_src" href="https://www.cs.umd.edu/sites/default/files/cs_logo.png" />
<link rel="canonical" href="https://www.cs.umd.edu/home" />
<link rel="shortlink" href="https://www.cs.umd.edu/home" />
<meta property="og:type" content="website" />
<meta property="og:url" content="https://www.cs.umd.edu/" />
<meta property="og:title" content="UMD Department of Computer Science" />
  <title>Welcome | UMD Department of Computer Science</title>
  <link type="text/css" rel="stylesheet" href="https://www.cs.umd.edu/sites/default/files/css/
css_lQaZfjVpwP_oGNqdtWCSpJT1EMqXdMiU84ekLLxQnc4.css" media="all" />
<link type="text/css" rel="stylesheet" href="https://www.cs.umd.edu/sites/default/files/css/
css_j7JZunwKIqwvzJFjFk1gxRXtNOhQFJhkDZ7rptpQrYI.css" media="all" />
<link type="text/css" rel="stylesheet" href="https://www.cs.umd.edu/sites/default/files/css/
css_aKOkUxCOQLcDgrXMyAhVibR1c6JIPg4m4_p5wbm6lHM.css" media="all" />
<link type="text/css" rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@3.4.1/dist/css/bootstrap.min.css"
media="all" />
<link type="text/css" rel="stylesheet" href="https://cdn.jsdelivr.net/npm/@unicorn-fail/drupal-bootstrap-styles@0.0.2/dist/
3.3.1/7.x-3.x/drupal-bootstrap.min.css" media="all" />
<link type="text/css" rel="stylesheet" href="https://www.cs.umd.edu/sites/default/files/css/
css_1D7zGf7DYakhSXPzaG8NghJjfba3LavXmQ7ZjhLLuPo.css" media="all" />
  <!-- HTML5 element support for IE6-8 -->
```

`os.exec` Family of Functions

Function name	Description
`execl``(` *path, arg0, arg1, ...* `)`	Executes the program *path* with the given arguments.
`execle``(` *path, arg0, arg1, ..., environment* `)`	Same as **`execl`**, but takes an additional argument—the *environment* in which the program executes. The environment is a dictionary with keys that are environment variables and with values that are environment variable values.
`execlp``(` *path, arg0, arg1, ...* `)`	Same as **`execl`**, but searches for executable *path* in **`os.environ[ "PATH" ].`**
`execv``(` *path, arguments* `)`	Same as **`execl`**, but arguments are contained in a single tuple or list.
`excve``(` *path, arguments, environment* `)`	Same as **`execle`**, but arguments are contained in a single tuple or list.
`execvp``(` *path, arguments* `)`	Same as **`execlp`**, but arguments are contained in a single tuple or list.
`execvpe``(` *path, arguments, environment* `)`	Same as **`execvp`**, but takes an additional argument, *environment*, in which the program executes.

Controlling Process Input /Output

- UNIX-system users can use multiple programs together to achieve a particular result (e.g., **`ls | sort`** produces a sorted list of a directory's contents)

Controlling Process Input /Output

```
#subpro.py
import os
import subprocess
# determine operating system, then set directory-listing and
reverse-sort commands
if os.name == "nt" or os.name == "dos":   # Windows system
    fileList = "dir /B"
    sortReverse = "sort /R"
elif os.name == "posix":   # UNIX-compatible system
    fileList = "ls -1"
    sortReverse = "sort -r"
else:
    sys.exit("OS not supported by this program.")

# obtain stdout of directory-listing command
dirOut = subprocess.Popen(fileList, shell=True,
stdout=subprocess.PIPE).stdout

# obtain stdout of reverse-sort command
sortOut = subprocess.Popen(sortReverse, shell=True,
stdin=subprocess.PIPE, stdout=subprocess.PIPE)

filenames = dirOut.read().decode()   # output from directory-listing
command
```

Determine OS to set system-specific directory-listing and reverse-sort commands

Executes command and obtains **stdout** stream

Executes command and obtains **stdout** and **stdin** streams

Returns directory-listing command's output

Controlling Process Input /Output

```
# display output from directory-listing command
print("Before sending to sort")
print("(Output from '%s'):" % fileList)
print(filenames)

# send output to stdin of sort command and get sorted output
sorted_output, _ = sortOut.communicate(input=filenames.encode())
sortOut.stdout.close()  # close stdout of sort command

# display sorted output
print("After sending to sort")
print("(Output from '%s'):" % sortReverse)
print(sorted_output.decode())  # output from sort command
```

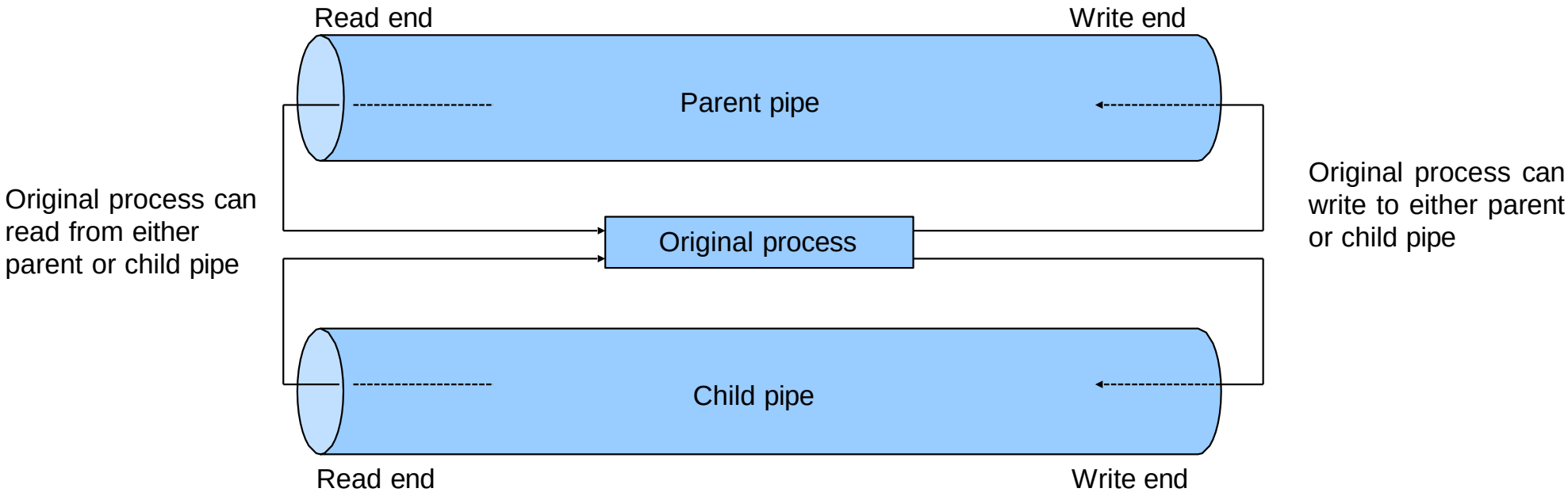
Controlling Process Input /Output

```
python subpro.py
Before sending to sort
(Output from 'ls -1'):
pid.py
subpro.py
systemfunc.py
wait.py
waitpid.py
waitpid1.py
webpage.py

After sending to sort
(Output from 'sort
-r'):
webpage.py
waitpid1.py
waitpid.py
wait.py
systemfunc.py
subpro.py
pid.py
```

Interprocess Communication

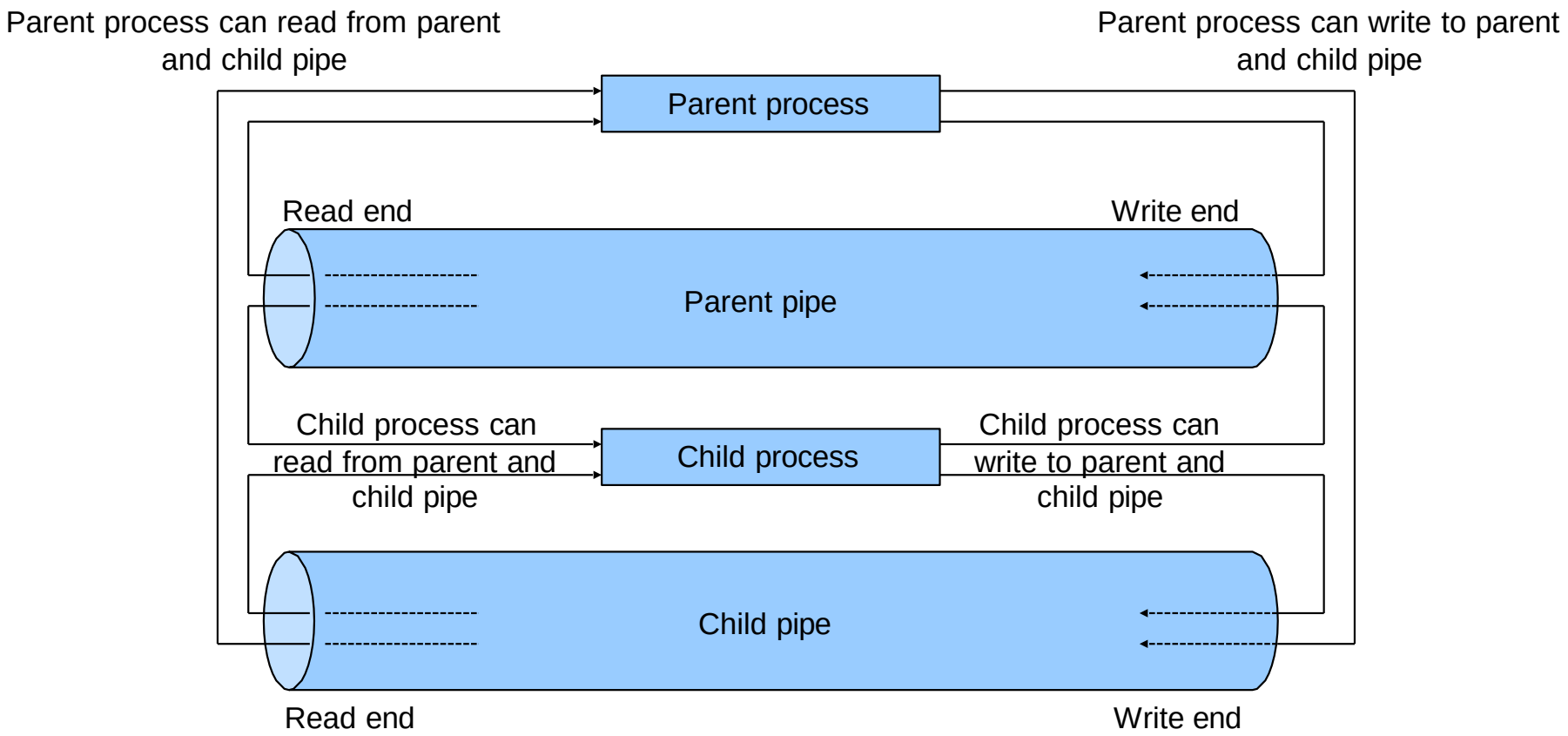
- Module **`os`** also provides interface to many interprocess communication (IPC) mechanisms
- Pipe, IPC mechanism, is a file-like object that provides a one-way communication channel
- Function **`os.pipe()`** creates a pipe and returns a tuple containing two file descriptors which provide read and write access to the pipe
- Functions **`os.read()`** and **`os.write()`** read and write the files associated with the file descriptors

Interprocess Communication



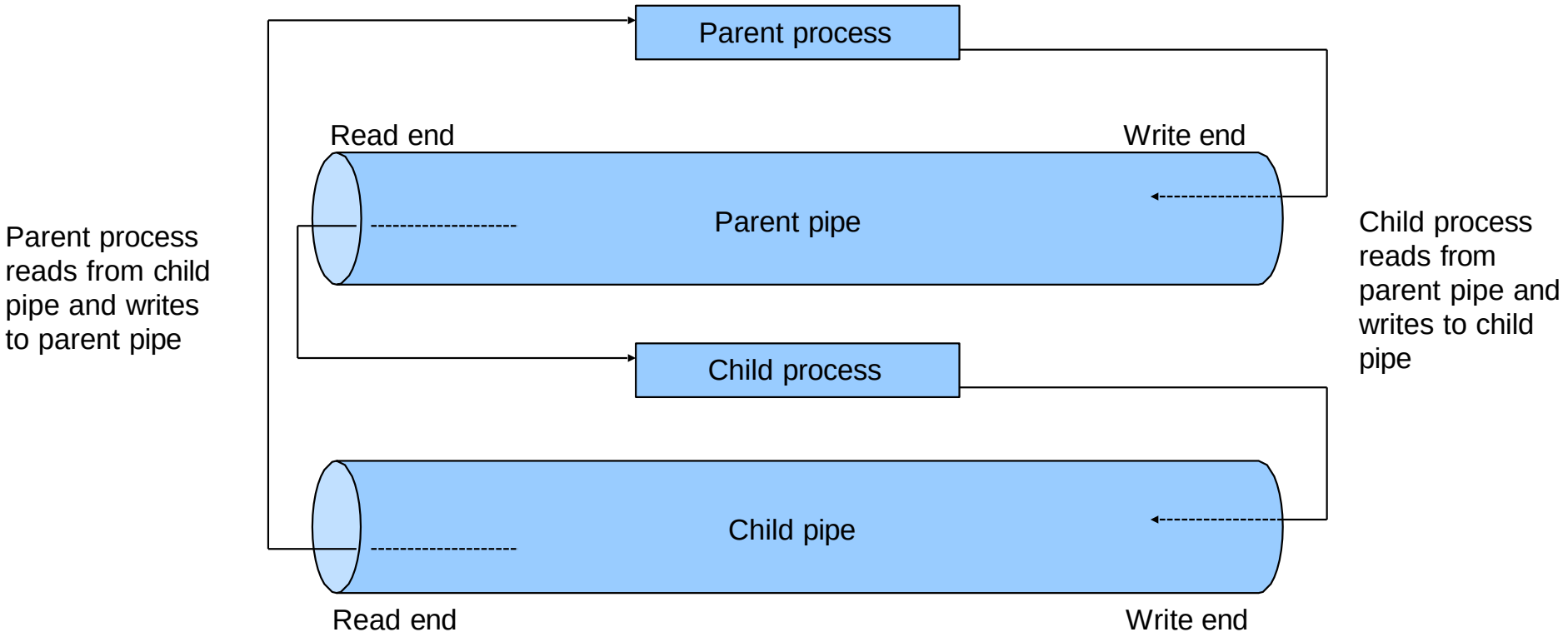
Initial stage: Original process can read and write to both pipes

Interprocess Communication



Intermediate stage: Parent and child processes can read and write to both pipes

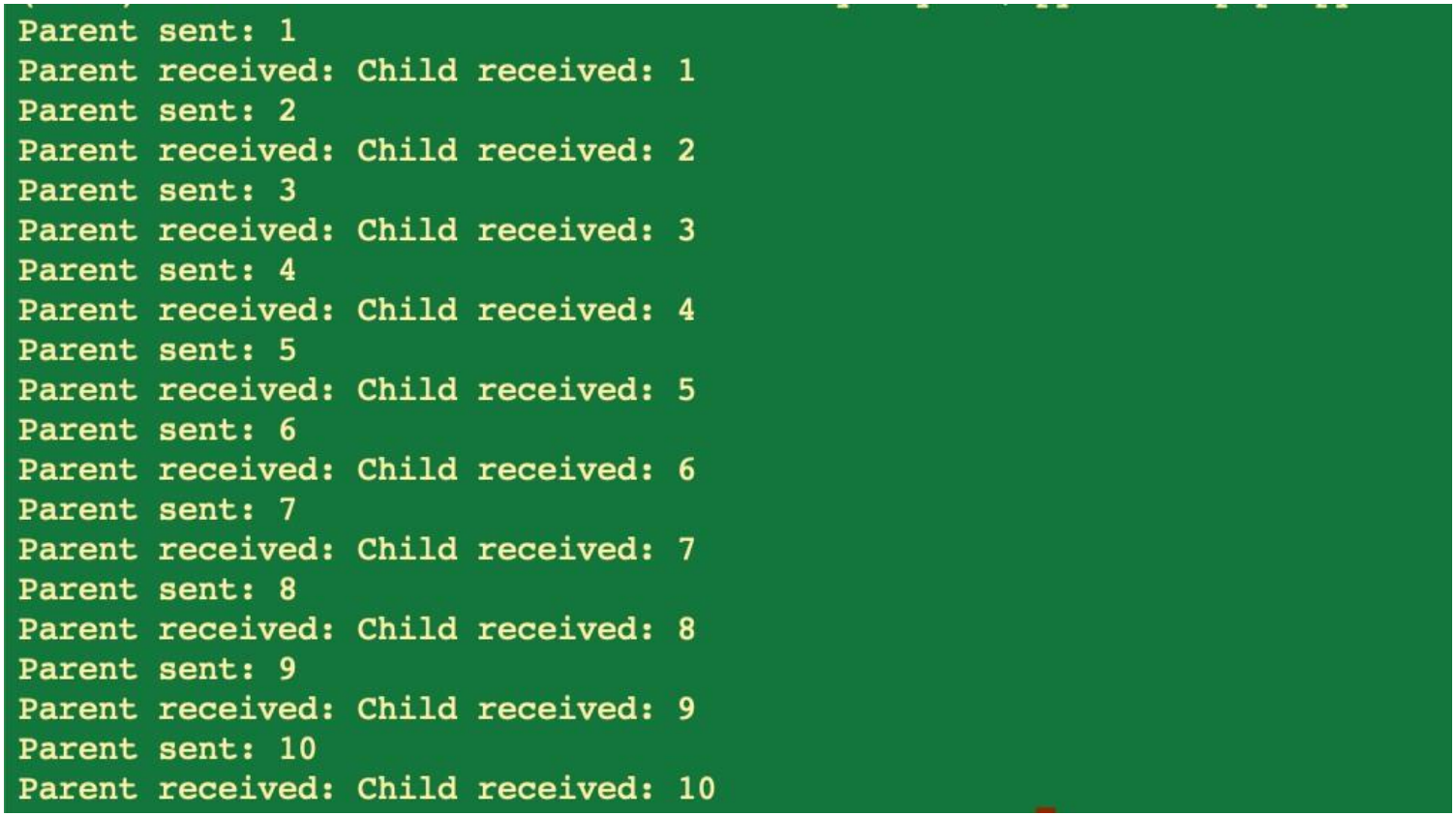
Interprocess Communication



Final stage: Parent and child processes close unneeded ends of the pipes

```python
# ospipe.py
# Using os.pipe to communicate with a child process.

    import os
    import sys

    # open parent and child read/write pipes
    fromParent, toChild = os.pipe()
    fromChild, toParent = os.pipe()

    # parent about to fork child process
    try:
       pid = os.fork()  # create child process
    except OSError:
       sys.exit( "Unable to create child process." )

    if pid != 0:  # am I parent process?

       # close unnecessary pipe ends
       os.close( toParent )
       os.close( fromParent )

       # write values from 1-10 to parent's write pipe and
       # read 10 values from child's read pipe
       for i in range( 1, 11 ):
          os.write( toChild, str( i ).encode())
          print(f"Parent sent: {i}")
          response = os.read(fromChild, 64)  # Read response from child
          print(f"Parent received: {response.decode()}")
```

```
31    else:  # Child process
32        # Close unnecessary pipe ends
33        os.close(toChild)
          os.close(fromChild)
34
35        # Read values from parent and send modified
response back
36
37        for _ in range(10):
38            value = os.read(fromParent, 64).decode()   #
Read value from parent
39            modified_value = f"Child received: {value}"
# Modify the received value
40            os.write(toParent, modified_value.encode())
# Send modified value back to parent
```


Parent sent: 1
Parent received: Child received: 1
Parent sent: 2
Parent received: Child received: 2
Parent sent: 3
Parent received: Child received: 3
Parent sent: 4
Parent received: Child received: 4
Parent sent: 5
Parent received: Child received: 5
Parent sent: 6
Parent received: Child received: 6
Parent sent: 7
Parent received: Child received: 7
Parent sent: 8
Parent received: Child received: 8
Parent sent: 9
Parent received: Child received: 9
Parent sent: 10
Parent received: Child received: 10

Signal Handling

- Processes can communicate with signals, which are messages that the OS delivers to programs asynchronously
- Signal processing: program receives a signal and performs an action based on that signal
- Signal handlers execute in response to signals
- Every Python program has default signal handlers

Signal Handling

```
#signalhandler.py
import time
import signal

def stop( signalNumber, frame ):
     global keepRunning
     keepRunning -= 1
     print ("Ctrl-C pressed; keepRunning is", keepRunning)

keepRunning = 3

# set the handler for SIGINT to be function stop
signal.signal( signal.SIGINT, stop )

while keepRunning:
    print "Executing..."
    time.sleep( 1 )

print "Program terminating..."
```

Signal Handling

```
Executing...
Executing...
Executing...
Executing...
Executing...
Executing...
Executing...
Executing...
Executing...
^CCtrl-C pressed; keepRunning is 2
Executing...
Executing...
Executing...
Executing...
çExecuting...
^CCtrl-C pressed; keepRunning is 1
Executing...
Executing...
^CCtrl-C pressed; keepRunning is 0
Program terminating...
```

Sending Signals

- Executing programs can send signals to other processes
- Function **`os.kill()`** sends a signal to a particular process identified by its *pid*

Sending Signals

```
1   # interprocesssignal.py
2   # Sending signals to child processes using kill
3
4   import os
5   import signal
6   import time
7   import sys
8
9   # handles both SIGALRM and SIGINT signals
10  def parentInterruptHandler( signum, frame ):
11     global pid
12     global parentKeepRunning
13
14     # send kill signal to child process and exit
15     os.kill( pid, signal.SIGKILL )   # send kill signal
16     print "Interrupt received. Child process killed."
17
18     # allow parent process to terminate normally
19     parentKeepRunning = 0
20
21  # set parent's handler for SIGINT
22  signal.signal( signal.SIGINT, parentInterruptHandler )
23
24  # keep parent running until child process is killed
25  parentKeepRunning = 1
```

Sending Signals

```
27    # parent ready to fork child process
28    try:
29       pid = os.fork()   # create child process
30    except OSError:
31. sys.exit( "Unable to create child process." )
32
33    if pid != 0:   # am I parent process?
34       while parentKeepRunning:
35          print "Parent running. Press Ctrl-C to terminate  child."
36          time.sleep( 1 )
37
38    elif pid == 0:   # am I child process?
39
40       # ignore interrupt in child process
41       signal.signal( signal.SIGINT, signal.SIG_IGN )
42
43       while 1:
44          print "Child still executing."
45          time.sleep( 1 )
46
47    print "Parent terminated child process."
48    print "Parent terminating normally."
```

Sending Signals

```
Parent running. Press Ctrl-C to terminate child.
Child still executing.
Child still executing.
Parent running. Press Ctrl-C to terminate child.
Child still executing.
Parent running. Press Ctrl-C to terminate child.
Child still executing.
Parent running. Press Ctrl-C to terminate child.
Child still executing.
Parent running. Press Ctrl-C to terminate child.
Child still executing.
Parent running. Press Ctrl-C to terminate child.
Child still executing.
Parent running. Press Ctrl-C to terminate child.
Child still executing.
Parent running. Press Ctrl-C to terminate child.
Child still executing.
Parent running. Press Ctrl-C to terminate child.
Child still executing.
Parent running. Press Ctrl-C to terminate child.
^CInterrupt received. Child process killed.
Parent terminated child process.
Parent terminating normally.
```

Multithreading in Python: The Hidden Truth

Outline

Introduction
Thread States: Life Cycle of a Thread
`threading.Thread` Example
Thread Synchronization
Producer/Consumer without Synchronization
Producer/Consumer with Synchronization
Producer/Consumer: Module `Queue`
Producer/Consumer: The Circular Buffer
Semaphores
Events

Process

- *A unit of work, for example, Jupyter notebook*
- *An OS can run multiple processes at the same time.*
- *By default Python interpreter executes instructions serially.*
- *The size of the datasets has in increased.*
- *The algorithms are more complex and need to process more, hence the need for multi-processing*

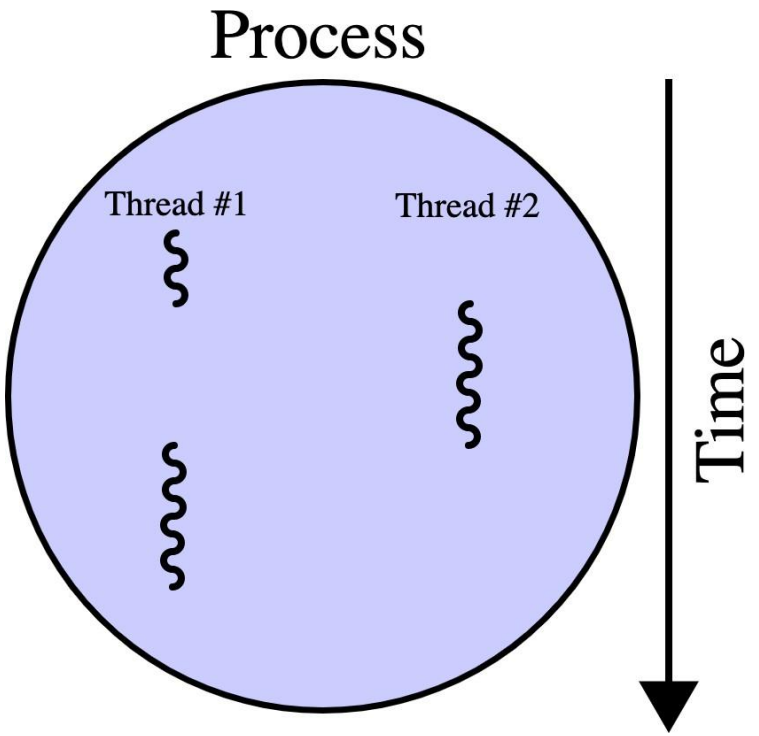
Parallel processing

- *It can be achieved in two ways: Multiprocessing and Threading*
- *Process: An instance of a program*
  Uses its own memory space

- *Threads: components of a process, which can run in parallel*
  - *Multiple threads*
  - *Share parent process memory space*

Processes and Threads

- *Threads live in the same memory space*
- *Processes have their separate memory space*
- *Spawning processes is slower than spawning threads.*
- *Sharing objects between threads is easier.*



Source

- *Inter-process communication between processes.*

Processes and Threads

- A process is a program in execution
- A thread is one line of execution within a process. A process may contain many threads
- Thread creation has much less overhead than process creation, especially in Windows
- Each thread has its own stack (local variables), but share global variables with the other threads
- Global variables allow threads to share information and communicate with one another
- Shared data introduces the need for synchronization – A can of worms

Introduction

- Threads often called “light-weight” process because operating systems generally require fewer resources to create and manage threads than to create and manage processes
- Python’s threading capabilities depend on whether the operating system supports multithreading
- Python’s **`threading`** module provides its multithreading capabilities

Thread States: Life Cycle of a Thread

- Python interpreter controls all program threads
- Python's global interpreter lock (GIL) ensures that the interpreter runs only one thread at any given time
- Thread is said to be in one of several thread states at any time
- Python programs can define threads by inheriting from class **`threading.Thread`** and overriding its functionality

THREAD STATES

Thread States

- Newly created thread begins its lifecycle in the born state
- Invoking the thread's **`start`** method places the thread in the ready state
- The thread's **`run`** method, which implements the tasks that the thread performs, obtains the GIL and enters the running state
- Thread enters the dead state when its **`run`** method returns or terminates for any reason (e.g., an uncaught exception)

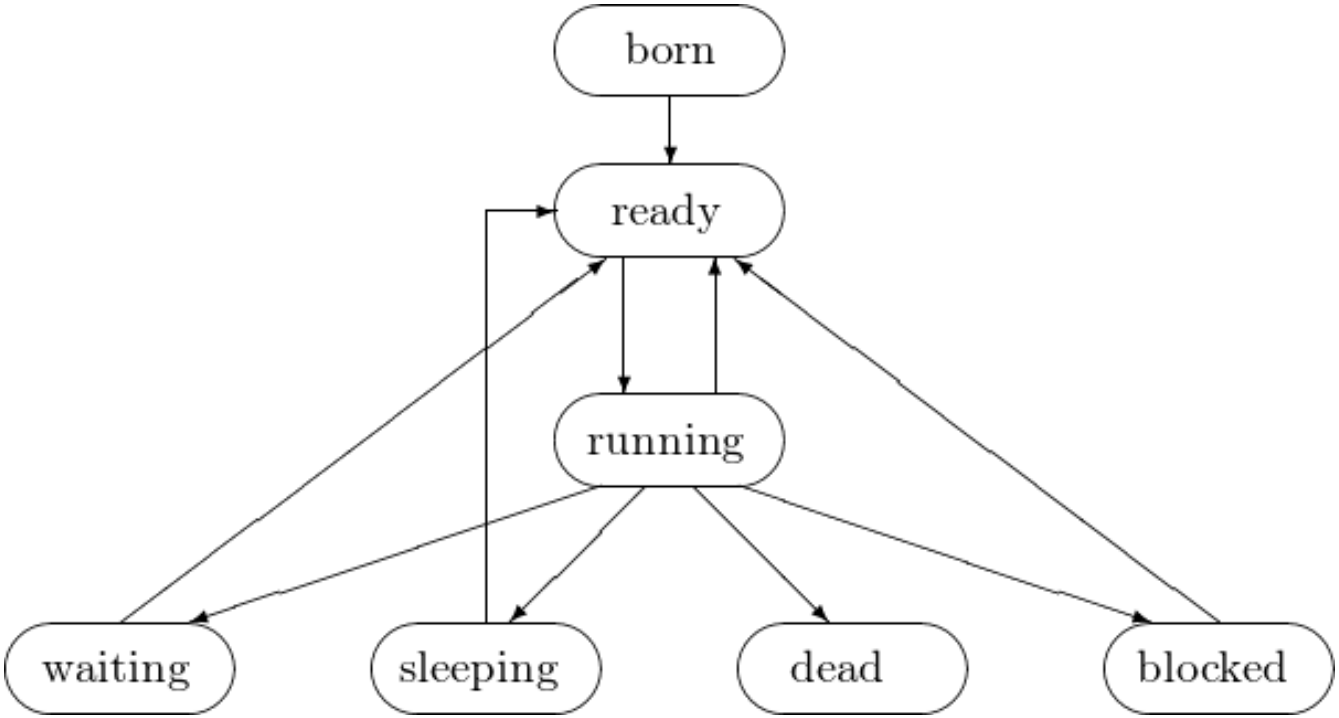
Thread States

- A running thread that calls another thread's **`join`** method forfeits the GIL and waits for the **`join`**ed thread to die before proceeding
- Thread enters the blocked state while it waits for a requested, unavailable resource (e.g. I/O device)
- When a running thread calls function **`time.sleep()`**, it releases the GIL and enters the sleeping state

Thread States

- Deadlock occurs when one or more threads wait forever for an event that cannot occur
- In indefinite postponement, one or more threads is delayed for some unpredictably long time

Thread States



State diagram showing the life cycle of a thread

Thread Information

- Module **`threading`** provides ways for a program to obtain information about its threads, including their current states
  - Function **`threading.currentThread()`** returns reference to currently running **`Thread`** object
  - Function **`threading.enumerate()`** returns list of currently active **`Thread`** objects
  - Function **`threading.activeCount()`** returns length of list returned by **`threading.enumerate()`**
  - Method **`isAlive()`** returns 1 if the **`Thread`** object's **`start`** method bas been invoked and the **`Thread`** object is not dead
  - Methods **`setName()`** and **`getName()`** allow programmer to set and get a **`Thread`** object's name, respectively

`threading.Thread` example

- Threads can be created by defining a class that derives from **`threading.Thread`** and instantiating objects of that class

Threads

```
# threads.py
# Multiple threads printing at different intervals.

    import threading
    import random
    import time

    class PrintThread( threading.Thread ):
       """Subclass of threading.Thread"""

       def __init__( self, threadName ):
          """Initialize thread, set sleep time, print data"""

          threading.Thread.__init__( self, name = threadName )
          self.sleepTime = random.randrange( 1, 6 )
          print "Name: %s; sleep: %d" % \
             ( self.getName(), self.sleepTime )

       # overridden Thread run method
       def run( self ):
          """Sleep for 1-5 seconds"""

          print "%s going to sleep for %s second(s)" % \
             ( self.getName(), self.sleepTime )
          time.sleep( self.sleepTime )
          print self.getName(), "done sleeping"

    thread1 = PrintThread( "thread1" )
    thread2 = PrintThread( "thread2" )
    thread3 = PrintThread( "thread3" )

    print "\nStarting threads"
    # Start the threads
    thread1.start()
    thread2.start()
    thread3.start()
```

Threads

```
Name: thread1; sleep: 3
Name: thread2; sleep: 4
Name: thread3; sleep: 1

Starting threads
thread1 going to sleep for 3 second(s)
thread2 going to sleep for 4 second(s)
thread3 going to sleep for 1 second(s)
thread3 done sleeping
thread1 done sleeping
thread2 done sleeping
```

THREAD SYNCHRONIZATION

Synchronisation

- Only one thread can update global data at a time.
- Multiple threads reading global data is allowed
- Critical code section – that section of the code which accesses the shared global data
- Single thread access to the critical section is easy, just acquire and release one lock

Synchronisation

- Multiple thread access to critical section is tricky

1. Known solutions to lots of synchronization problems
2. Hardest part is framing the problem in terms of a known synchronization problem: producers and consumers, readers and writers, sleepy barber, smokers, one lane bridge, etc…
3. Take Operating Systems class or study parallel programming
4. Some Python modules implement known problem solutions

Thread Synchronisation

- Sections of code that access shared data are referred to as critical sections
- Mutual exclusion or thread synchronization ensures a thread accessing the shared data excludes all other threads from doing so simultaneously
- Module threading provides several thread-synchronization mechanisms (e.g., locks and condition variables)

Thread Synchronisation

- Class threading.RLock creates lock objects
- Method acquire() causes lock to enter locked state
- Only one thread may acquire lock at a time so system places any other threads who attempt to acquire lock in blocked state
- When the thread that owns the lock invokes method release(), the lock enters the unlocked state and blocked threads receive a notification so one can acquire the lock

Condition Variables

- Condition variables monitor the state of an object or notify a thread when an event occurs

1. Created with class threading.Condition
2. Provides acquire() and release() method because condition variable contains underlying locks
3. Method wait() causes calling thread to release lock and block until it is awakened again
4. Method notify() wakes up one thread waiting on the condition variable
5. Method notifyAll() wakes up all waiting threads

Producer/Consumer w/out Synchronization

- Producer portion of application generates data
- Consumer portion of application uses that data
- Producer thread calls produce method to generate data and place it into a shared memory region called a buffer
- Consumer thread calls consumer method to read data from the buffer

```
# fig19_03.py
# Multiple threads modifying shared object.

from UnsynchronizedInteger import UnsynchronizedInteger
from ProduceInteger import ProduceInteger
from ConsumeInteger import ConsumeInteger

# initialize integer and threads
number = UnsynchronizedInteger()
producer = ProduceInteger( "Producer", number, 1, 4 )
consumer = ConsumeInteger( "Consumer", number, 4 )

print("Starting threads...\n")

# start threads
producer.start()
consumer.start()

# wait for threads to terminate
producer.join()
consumer.join()
print("\nAll threads have terminated.")
```

Create shared UnsynchronisedInteger object number

Instantiate ProduceInteger thread object

Create ConsumeInteger thread object

invoke thread object's start methods

Ensures that main program waits for both threads to terminate before proceeding

```
Starting threads...

Consumer reads -1
Consumer reads -1
Producer writes 1
Consumer reads 1
Producer writes 2
Consumer reads 2
Consumer read values totaling: 1.
Terminating Consumer.
Producer writes 3
Producer writes 4
Producer done producing.
Terminating Producer.

All threads have terminated.
```

```
Starting threads...

Producer writes 1
Producer writes 2
Producer writes 3
Consumer reads 3
Producer writes 4
Producer done producing.
Terminating Producer.
Consumer reads 4
Consumer reads 4
Consumer reads 4
Consumer read values totaling: 15.
Terminating Consumer.

All threads have terminated.
```

```
Starting threads...

Producer writes 1
Consumer reads 1
Producer writes 2
Consumer reads 2
Producer writes 3
Consumer reads 3
Producer writes 4
Producer done producing.
Terminating Producer.
Consumer reads 4
Consumer read values totaling: 10.
Terminating Consumer.

All threads have terminated
```

```python
# ProduceInteger.py
# Integer-producing class.

import threading
import random
import time

class ProduceInteger( threading.Thread ):
    """Thread to produce integers"""

    def __init__( self, threadName, sharedObject, begin, end ):
        """Initialize thread, set shared object"""

        threading.Thread.__init__( self, name = threadName )
        self.sharedObject = sharedObject
        self.begin = begin
        self.end = end

    def run( self ):
        """Produce integers in given range at random intervals"""

        for i in range( self.begin, ( self.end + 1 ) ):
            time.sleep( random.randrange( 4 ) )
            self.sharedObject.set( i )

        print("{} done producing.".format(self.name))
        print("Terminating {}.".format(self.name))
```

Subclass of threading.Thread

Set variable sharedObject to refer to object shared by producer and consumer

Loops from begin to end

Set shared object

Prints message indicating that run method has terminated

```python
# ConsumeInteger.py
# Integer-consuming queue.

import threading
import random
import time

class ConsumeInteger( threading.Thread ):
    """Thread to consume integers"""

    def __init__( self, threadName, sharedObject, amount ):
        """Initialize thread, set shared object"""

        threading.Thread.__init__( self, name = threadName )
        self.sharedObject = sharedObject
        self.amount = amount

    def run( self ):
        """Consume given amount of values at random time intervals"""

        sum = 0   # total sum of consumed values

        # consume given amount of values
        for i in range( self.amount ):
            time.sleep( random.randrange( 4 ) )
            sum += self.sharedObject.get()

        print("{} read values totaling: {}.".format( self.name, sum ))
        print("Terminating {}.".format( self.name ))
```

Refers to the shared UnsynchronizedInteger object

Sums values of the shared UnsynchronizedInteger object

Print total sum of consumed values

```
# UnsynchronizedInteger.py
# Unsynchronized access to an integer.

import threading

class UnsynchronizedInteger:
    """Class that provides unsynchronized access an integer"""

    def __init__( self ):
        """Initialize integer to -1"""

        self.buffer = -1

    def set( self, newNumber ):
        """Set value of integer"""

        print("{} writes {} \n".format
            ( threading.current_thread().name, newNumber ))
        self.buffer = newNumber

    def get( self ):
        """Get value of integer"""

        tempNumber = self.buffer
        print("{} reads {} \n".format
            ( threading.current_thread().name, tempNumber ))

        return tempNumber
```

Provides unsynchronised access to an integer

Initialise integer to -1

Sets integer to specified value

Returns value of integer

Use temporary variable to ensure that program prints correct value

Producer/Consumer w/ Synchronization

- Producer and consumer can access shared memory cell with synchronization

1. Ensures that the consumer consumes each value exactly once
2. Ensures that consumer waits for producer to execute first

```
# fig19_07.py
# Multiple threads modifying shared object.

from SynchronizedInteger import SynchronizedInteger
from ProduceInteger import ProduceInteger
from ConsumeInteger import ConsumeInteger

# initialize number and threads
number = SynchronizedInteger()
producer = ProduceInteger( "Producer", number, 1, 4 )
consumer = ConsumeInteger( "Consumer", number, 4 )

print("Starting threads...\n")

print("{: <35} {: <9} {}\n".format
      ( "Operation" , "Buffer", "Occupied Count" ))
number.displayState( "Initial state" )

# start threads
producer.start()
consumer.start()

# wait for threads to terminate
producer.join()
consumer.join()

print("\nAll threads have terminated.")
```

Create SynchronizedInteger object

Start producer and consumer threads

```
 Starting threads...

Operation                          Buffer   Occupied Count

Initial state                      -1        0

Producer writes 1                  1

Consumer reads 1                   1         0

Consumer tries to read.
Buffer empty. Consumer waits.      1         0

Producer writes 2                  2         1

Consumer reads 2                   2         0

Producer writes 3                  3         1

Producer tries to write.
Buffer full. Producer waits.       3         1

Consumer reads 3                   3         0

Producer writes 4                  4         1

Producer done producing.
Terminating Producer.
Consumer reads 4                   4         0

Consumer read values totaling: 10.
Terminating Consumer.

All threads have terminated.
```

```
Starting threads...

Operation                          Buffer   Occupied Count

Initial state                      -1        0

Producer writes 1                  1         1

Consumer reads 1                   1         0

Producer writes 2                  2         1

Consumer reads 2                   2         0

Producer writes 3                  3         1

Consumer reads 3                   3         0


Producer writes 4                  4         1

Producer done producing.
Terminating Producer.
Consumer reads 4                   4         0

Consumer read values totaling: 10.
Terminating Consumer.

All threads have terminated.
```

```
# SynchronizedInteger.py
# Synchronized access to an integer with condition variable.

import threading

class SynchronizedInteger:
    """Class that provides synchronized access an integer"""

    def __init__( self ):
        """Initialize integer, buffer count and condition variable"""

        self.buffer = -1
        self.occupiedBufferCount = 0   # number of occupied buffers
        self.threadCondition = threading.Condition()

    def set( self, newNumber ):
        """Set value of integer--blocks until lock acquired"""

        # block until lock released then acquire lock
        self.threadCondition.acquire()

        # while not producer's turn, release lock and block
        while self.occupiedBufferCount == 1:
            print("{} tries to write.".format(
               threading.current_thread().name))
            self.displayState( "Buffer full. " + \
               threading.current_thread().name + " waits." )
            self.threadCondition.wait()

         # (lock has now been re-acquired)

        self.buffer = newNumber          # set new buffer value
        self.occupiedBufferCount += 1    # allow consumer to consume


        self.displayState( "{} writes {}".format
            ( threading.current_thread().name, newNumber ))

        self.threadCondition.notify()    # wake up a waiting thread
        self.threadCondition.release()   # allow lock to be acquired
```

Provides synchronized access to an integer

Sets value of integer, blocking until lock acquired

Method acquire blocks until lock available

Block until producer's turn

Set new buffer value after lock is re-acquired

Allow lock to be acquired

```
def get( self ):
    """Get value of integer--blocks until lock acquired"""

    # block until lock released then acquire lock
    self.threadCondition.acquire()

    # while producer's turn, release lock and block
    while self.occupiedBufferCount == 0:
        print("{} tries to read.".format(
           threading.current_thread().name))
        self.displayState( "Buffer empty. " + \
           threading.current_thread().name + " waits." )
        self.threadCondition.wait()

     # (lock has now been re-acquired)

    tempNumber = self.buffer
    self.occupiedBufferCount -= 1 # allow producer to produce

    self.displayState( "{} reads {}".format
        ( threading.current_thread().name, tempNumber ) )

    self.threadCondition.notify()    # wake up a waiting thread
    self.threadCondition.release()   # allow lock to be acquired

    return tempNumber

def displayState( self, operation ):
    """Display current state"""

    print("{: <35} {: <9} {}\n".format
        ( operation, self.buffer, self.occupiedBufferCount ))
```

Return value of integer when lock acquired

Release lock and block until consumer's turn

Display buffer state

Displays current state of buffer

Producer/Consumer: Module Queue

- Module Queue defines class Queue, which is a synchronized implementation of a queue
- Queue constructor accepts optional argument maxsize
- Consumer consumes value by calling Queue method get, which removes and returns item from the head of the queue
- Producer produces value by calling Queue method put, which inserts item at tail of queue

```
# fig19_09.py
# Multiple threads producing/consuming values.

from Queue import Queue
from ProduceToQueue import ProduceToQueue
from ConsumeFromQueue import ConsumeFromQueue

# initialize number and threads
queue = Queue()
producer = ProduceToQueue( "Producer", queue )
consumer = ConsumeFromQueue( "Consumer", queue )

print("Starting threads...\n")

# start threads
producer.start()
consumer.start()

# wait for threads to terminate
producer.join()
consumer.join()

print("\nAll threads have terminated.")
```

Class Queue is synchronized queue implementation

Instantiate infinite-size queue

Start producer and consumer threads

Main program waits for consumer and producer threads to terminate

```
# ProduceToQueue.py
# Integer-producing class.

import threading
import random
import time

class ProduceToQueue( threading.Thread ):
    """Thread to produce integers"""

    def __init__( self, threadName, queue ):
        """Initialize thread, set shared queue"""

        threading.Thread.__init__( self, name = threadName )
        self.sharedObject = queue

    def run( self ):
        """Produce integers in range 11-20 at random intervals"""

        for i in range( 11, 21 ):
            time.sleep( random.randrange( 4 ) )
            print("{} adding {} to queue".format( self.name, i ))
            self.sharedObject.put( i )

        print(self.name, "finished producing values")
        print("Terminating", self.name)
```

Producer thread uses synchronized queue

Refers to shared synchronized Queue obje

Place item at tail of the queue

```
# ConsumeFromQueue.py
# Integer-consuming queue.


import threading
import random
import time

class ConsumeFromQueue( threading.Thread ):
    """Thread to consume integers"""

    def __init__( self, threadName, queue ):
        """Initialize thread, set shared queue"""

        threading.Thread.__init__( self, name = threadName )
        self.sharedObject = queue

    def run( self ):
        """Consume 10 values at random time intervals"""

        sum = 0             # total sum of consumed values
        current = 10        # last value retrieved

        # consume 10 values
        for i in range( 10 ):
            time.sleep( random.randrange( 4 ) )
            print("{} attempting to read {}...".format
               ( self.name, current + 1 ))
            current = self.sharedObject.get()
            print("{} read {}".format( self.name, current ))
            sum += current

        print("{} retrieved values totaling: {}".format
            ( self.name, sum ))
        print("Terminating", self.name)
```

Consumer thread using synchronized Queue object

Refers to shared Queue object

Get item from head of queue

```
Starting threads...

Producer adding 11 to queue
Producer adding 12 to queue
Consumer attempting to read 11...
Consumer read 11
Consumer attempting to read 12...
Consumer read 12
Producer adding 13 to queue
Consumer attempting to read 13...
Consumer read 13
Producer adding 14 to queue
Consumer attempting to read 14...
Consumer read 14
Consumer attempting to read 15...
Producer adding 15 to queue
Consumer read 15
Consumer attempting to read 16...
Producer adding 16 to queue
Consumer read 16
Consumer attempting to read 17...
Producer adding 17 to queue
Consumer read 17
Producer adding 18 to queue
Producer adding 19 to queue
Consumer attempting to read 18...
Consumer read 18
Consumer attempting to read 19...
Consumer read 19
Producer adding 20 to queue
Producer finished producing values
Terminating Producer
Consumer attempting to read 20...
Consumer read 20
Consumer retrieved values totaling: 155
Terminating Consumer
All threads have terminated.
```

Producer/Consumer: The Circular Buffer

- Circular buffer provides extra buffers into which the producer can place values and from which the consumer can retrieve those values

```
# fig19_12.py
# Show multiple threads modifying shared object.

# from CircularBuffer import CircularBuffer
# from ProduceInteger import ProduceInteger
# from ConsumeInteger import ConsumeInteger

# initialize number and threads
buffer = CircularBuffer()
producer = ProduceInteger( "Producer", buffer, 11, 20 )
consumer = ConsumeInteger( "Consumer", buffer, 10 )

print("Starting threads...\n")

buffer.displayState()

# start threads
producer.start()
consumer.start()

# wait for threads to terminate
producer.join()
consumer.join()

print("\nAll threads have terminated.")
```

Create CircularBuffer object

Start producer and consumer threads

```
# CircularBuffer.py
# Synchronized circular buffer of integer values

import threading

class CircularBuffer:

    def __init__( self ):
        """Set buffer, count, locations and condition variable"""

        # each element in list is a buffer
        self.buffer = [ -1, -1, -1 ]

        self.occupiedBufferCount = 0   # count of occupied buffers
        self.readLocation = 0          # current reading index
        self.writeLocation = 0         # current writing index

        self.threadCondition = threading.Condition()
```

Three-element integer list that represents the circular buffer

Condition variable protects access to circular buffer

```
def set( self, newNumber ):
    """Set next buffer index value--blocks until lock acquired"""

    # block until lock released then acquire lock
    self.threadCondition.acquire()

    # while all buffers are full, release lock and block
    while self.occupiedBufferCount == len( self.buffer ):
        print("All buffers full. {} waits.".format(threading.current_thread().name))
        self.threadCondition.wait()

    # (there is an empty buffer, lock has been re-acquired)
    # place value in writeLocation of buffer
    # print string indicating produced value
    self.buffer[ self.writeLocation ] = newNumber
    print("{} writes {} ".format( threading.current_thread().name, newNumber ))

    # produced value, so increment number of occupied buffers
    self.occupiedBufferCount += 1

    # update writeLocation for future write operation
    # add current state to output
    self.writeLocation = ( self.writeLocation + 1 ) % len( self.buffer )
    self.displayState()

    self.threadCondition.notify()    # wake up a waiting thread
    self.threadCondition.release()   # allow lock to be acquired
```

Release lock and block while all buffers are full

Place value at write index

Update write index for future write operations

```
def get( self ):
    """Get next buffer index value--blocks until lock acquired"""

    # block until lock released then acquire lock
    self.threadCondition.acquire()

    # while all buffers are empty, release lock and block
    while self.occupiedBufferCount == 0:
        print("All buffers empty. {} waits.".format(threading.current_thread().name))
        self.threadCondition.wait()

    # (there is a full buffer, lock has been re-acquired)

    # obtain value at current readLocation
    # print string indicating consumed value
    tempNumber = self.buffer[ self.readLocation ]
    print("{} reads {} ".format( threading.current_thread().name, tempNumber ))

    # consumed value, so decrement number of occupied buffers
    self.occupiedBufferCount -= 1

    # update readLocation for future read operation
    # add current state to output
    self.readLocation = ( self.readLocation + 1 ) % len( self.buffer )
    self.displayState()

    self.threadCondition.notify()    # wake up a waiting thread
    self.threadCondition.release()   # allow lock to be acquired

    return tempNumber
```

Release lock and block while buffers are empty

Obtain and print value at current read index

Update read index for future read operations

```
def displayState( self ):Display current circular buffer state
    """Display current state"""

    # display first line of state information
    print("(buffers occupied: {})".format(self.occupiedBufferCount))
    print("buffers: ")

    for item in self.buffer:
        print(" {}  ".format(item))


    # display second line of state information
    print("\n         ",)

    for item in self.buffer:
        print("---- ")

    # display third line of state information
    print("\n         ")

    for i in range( len( self.buffer ) ):

        if ( i == self.writeLocation ) and ( self.writeLocation == self.readLocation ):
            print(" WR  ")
        elif ( i == self.writeLocation ):
            print(" W   ")
        elif ( i == self.readLocation ):
            print("  R  ")
        else:
            print("     ")

    print("\n")
```

```
 Starting threads...

(buffers occupied: 0)
buffers:   -1    -1    -1
       ----  ----  ----
        WR

Producer writes 11  (buffers occupied: 1)
buffers:   11    -1    -1
       ----  ----  ----
         R    W

Consumer reads 11  (buffers occupied: 0)
buffers:   11    -1    -1
       ----  ----  ----
            WR


All buffers empty. Consumer waits.
Producer writes 12  (buffers occupied: 1)
buffers:   11    12    -1
       ----  ----  ----
            R    W

Consumer reads 12  (buffers occupied: 0)
buffers:   11    12    -1
       ----  ----  ----
               WR

All buffers empty. Consumer waits.
Producer writes 13  (buffers occupied: 1)
buffers:   11    12    13
       ----  ----  ----
        W           R
```

```
Consumer reads 13  (buffers occupied: 0)
buffers:   11    12    13
        ----  ----  ----
         WR

All buffers empty. Consumer waits.
Producer writes 14  (buffers occupied: 1)
buffers:   14    12    13
        ----  ----  ----
          R    W

Consumer reads 14  (buffers occupied: 0)
buffers:   14    12    13
        ----  ----  ----
             WR

Producer writes 15  (buffers occupied: 1)
buffers:   14    15    13
        ----  ----  ----
              R    W

Producer writes 16  (buffers occupied: 2)
buffers:   14    15    16
        ----  ----  ----
         W      R

Consumer reads 15  (buffers occupied: 1)
buffers:   14    15    16
        ----  ----  ----
         W           R

Producer writes 17  (buffers occupied: 2)
buffers:   17    15    16
        ----  ----  ----
             W     R
```

```
Producer writes 18  (buffers occupied: 3)
buffers:  17   18   16
       ---- ---- ----
                 WR

All buffers full. Producer waits.
Consumer reads 16  (buffers occupied: 2)
buffers:  17   18   16
       ---- ---- ----
        R         W

Producer writes 19  (buffers occupied: 3)
buffers:  17   18   19
       ---- ---- ----
        WR

All buffers full. Producer waits.
Consumer reads 17  (buffers occupied: 2)
buffers:  17   18   19
       ---- ---- ----
        W    R

Producer writes 20  (buffers occupied: 3)
buffers:  20   18   19
       ---- ---- ----
             WR

Producer done producing.
Terminating Producer.

Consumer reads 18  (buffers occupied: 2)
buffers:  20   18   19
       ---- ---- ----
             W    R
```

```
Consumer reads 19  (buffers occupied: 1)
buffers:   20    18    19
          ----  ----  ----
           R    W

Consumer reads 20  (buffers occupied: 0)
buffers:   20    18    19
          ----  ----  ----
                WR

Consumer read values totaling: 155.
Terminating Consume\r.

All threads have terminated
```

Semaphores

- Semaphore: variable that controls access to common resource or critical section
- Maintains a counter that specifies number of threads that can use the resource or enter the critical section simultaneously
- Counter decremented each time a thread acquires the semaphore
- When counter reaches zero, semaphore blocks other threads from accessing semaphore until it has been released by another thread

```
# fig19_14.py
# Semaphore to control access to a critical section.

import threading
import random
import time

class SemaphoreThread( threading.Thread ):
    """Class using semaphores"""

    availableTables = [ "A", "B", "C", "D", "E" ]

    def __init__( self, threadName, semaphore ):
        """Initialize thread"""

        threading.Thread.__init__( self, name = threadName )
        self.sleepTime = random.randrange( 1, 6 )

        # set the semaphore as a data attribute of the class
        self.threadSemaphore = semaphore

    def run( self ):
        """Print message and release semaphore"""

        # acquire the semaphore
        self.threadSemaphore.acquire()

        # remove a table from the list
        table = SemaphoreThread.availableTables.pop()
        print("{} entered; seated at table {}.".format( self.name, table ))
        print(SemaphoreThread.availableTables)

        time.sleep( self.sleepTime )   # enjoy a meal

        # free a table
        print("   {} exiting; freeing table {}.".format( self.name, table ))
        SemaphoreThread.availableTables.append( table )
        print(SemaphoreThread.availableTables)

        # release the semaphore after execution finishes
        self.threadSemaphore.release()

threads = []  # list of threads

# semaphore allows five threads to enter critical section
threadSemaphore = threading.Semaphore(len(SemaphoreThread.availableTables))

# create ten threads
for i in range( 1, 11 ):
    threads.append( SemaphoreThread( "thread" + str( i ), threadSemaphore ) )

# start each thread
for thread in threads:
    thread.start()
```

Class using semaphores

Set semaphore as a data attribute

Acquire the semaphore

Remove a table from the list

Free table by appending it to list

Release semaphore after critical section executes

Create threading.Semaphore and set internal counter to five

Start ten threads

```
thread1 entered; seated at table E. ['A', 'B', 'C', 'D']
thread2 entered; seated at table D. ['A', 'B', 'C']
thread3 entered; seated at table C. ['A', 'B']
thread4 entered; seated at table B. ['A']
thread5 entered; seated at table A. []
  thread2 exiting; freeing table D. ['D']
thread6 entered; seated at table D. []
  thread1 exiting; freeing table E. ['E']
thread7 entered; seated at table E. []
  thread3 exiting; freeing table C. ['C']
thread8 entered; seated at table C. []
  thread4 exiting; freeing table B. ['B']
thread9 entered; seated at table B. []
  thread5 exiting; freeing table A. ['A']
thread10 entered; seated at table A. []
  thread7 exiting; freeing table E. ['E']
  thread8 exiting; freeing table C. ['E', 'C']
  thread9 exiting; freeing table B. ['E', 'C', 'B']
  thread10 exiting; freeing table A. ['E', 'C', 'B', 'A']
  thread6 exiting; freeing table D. ['E', 'C', 'B', 'A', 'D']
```

Events

- Module **threading** defines class **Event**, which is useful for thread communication
- **Event** objects have an internal, true or false flag
- One or more threads call the **Event** object's **wait** method to block until the event occurs
- When the event occurs, the blocked thread or threads are awakened in the order that they arrived and resume execution

```
# fig19_15.py
# Event objects.

import threading
import random
import time

class VehicleThread( threading.Thread ):
    """Class representing a motor vehicle at an intersection"""

    def __init__( self, threadName, event ):
        """Initializes thread"""

        threading.Thread.__init__( self, name = threadName )

        # ensures that each vehicle waits for a green light
        self.threadEvent = event

    def run( self ):
        """Vehicle waits unless/until light is green"""

        # stagger arrival times
        time.sleep( random.randrange( 1, 10 ) )

        # prints arrival time of car at intersection
        print("{} arrived at {}".format( self.name, time.ctime( time.time() ) ) )

        # flag is false until light is green
        self.threadEvent.wait()

        # displays time that car departs intersection
        print("{} passes through intersection at {}".format
            ( self.name, time.ctime( time.time() ) ) )

greenLight = threading.Event()
vehicleThreads = []

# creates and starts ten Vehicle threads
for i in range( 1, 11 ):
    vehicleThreads.append( VehicleThread( "Vehicle" + str( i ), greenLight ) )

    for vehicle in vehicleThreads:
        vehicle.start()

    while threading.active_count() > 1:

        # sets the Event's flag to false -- block all incoming vehicles
        greenLight.clear()
        print("RED LIGHT! at", time.ctime( time.time() ) )
        time.sleep( 3 )

        # sets the Event's flag to true -- awaken all waiting vehicles
        print("GREEN LIGHT! at", time.ctime( time.time() ) )
        greenLight.set()
        time.sleep( 1 )
```

VehicleThread objects represent motor vehicle at an intersection

Each thread has Event object as data attribute

Thread is in blocked state until light is green

Displays message after Event flag becomes true

Instantiate Event object

Start eleven VehicleThread objects

Set Event object's flag to false

Set Event object's flag to true

```
RED LIGHT! at Fri Dec 21 18:10:44 2001
Vehicle7 arrived at Fri Dec 21 18:10:45 2001
Vehicle2 arrived at Fri Dec 21 18:10:46 2001
Vehicle8 arrived at Fri Dec 21 18:10:46 2001
GREEN LIGHT! at Fri Dec 21 18:10:47 2001
Vehicle7 passes through intersection at Fri Dec 21 18:10:47 2001
Vehicle2 passes through intersection at Fri Dec 21 18:10:47 2001
Vehicle8 passes through intersection at Fri Dec 21 18:10:47 2001
Vehicle1 arrived at Fri Dec 21 18:10:48 2001
Vehicle1 passes through intersection at Fri Dec 21 18:10:48 2001
Vehicle9 arrived at Fri Dec 21 18:10:48 2001
Vehicle9 passes through intersection at Fri Dec 21 18:10:48 2001
Vehicle10 arrived at Fri Dec 21 18:10:48 2001
Vehicle10 passes through intersection at Fri Dec 21 18:10:48 2001
RED LIGHT! at Fri Dec 21 18:10:48 2001
Vehicle4 arrived at Fri Dec 21 18:10:50 2001
Vehicle5 arrived at Fri Dec 21 18:10:50 2001
Vehicle6 arrived at Fri Dec 21 18:10:51 2001
GREEN LIGHT! at Fri Dec 21 18:10:51 2001
Vehicle4 passes through intersection at Fri Dec 21 18:10:51 2001
Vehicle5 passes through intersection at Fri Dec 21 18:10:51 2001
Vehicle6 passes through intersection at Fri Dec 21 18:10:51 2001
RED LIGHT! at Fri Dec 21 18:10:52 2001
Vehicle3 arrived at Fri Dec 21 18:10:53 2001
GREEN LIGHT! at Fri Dec 21 18:10:55 2001
Vehicle3 passes through intersection at Fri Dec 21 18:10:55 2001
```

Summary

Process

- A unit of work, for example, Jupyter notebook
- An OS can run multiple processes at the same time.
- By default Python interpreter executes instructions serially
- The size of the datasets has in increased
- The algorithms are more complex and need to process more, hence the need for multi-processing

Parallel Processing

- To speed up a process we want to split it to distribute across many CPUs
- Faster or/and efficiently
- Many tasks are suited for parallel processing, for example matrix multiplication
- A process can have multiple threads

Parallel Processing

- It can be achieved in two ways: **Multiprocessing** and **Threading**

- **Process**: An instance of a program
- Uses its own memory space

- **Threads**: components of a process, which can run in parallel
- Multiple threads
- Share parent process memory space

Cons of Parallel Processing

- **Race Condition**
- For threads same memory and access to variables
- To avoid, use mutex (mutual exclusion) lock around code
- **Starvation**: A thread is denied access to a resource for a long duration
- **Deadlock**: Mutex overuse can cause deadlocks. A thread has to wait for another thread to release a lock

- **Livelock**: threads keep running in a loop but don’t make any progress

Threading

- Use threading if network bound and multiprocessing if its CPU bound
- Threading is perfect
- For I/O operations such as web scraping
- GUI programs, for example one for text editing, another for recording and a third one to do spell-checking
- Tensorflow uses thread pool to transform data in parallel

Multiprocessing

- Useful when the program is CPU intensive and not dependent on IO or user interaction
- For example, processing numbers
- Pytorch Dataloaded loads data into GPU